
Hissab

Arithmetic Expression Evaluator

Technical Architecture Report

Activity 4 — Jakarta EE Component-Based Application

Author(s) HIMEUR Idris, DIAR Adem

Class SIQ1

Academic Year 2025/2026

May 16, 2026

Contents

1	Introduction	2
1.1	Key Features	2
2	Environment & Technology Stack	2
2.1	Infrastructure Setup	3
3	Architecture Overview	4
4	Development View (Class Diagram)	4
4.1	Package Structure	4
4.2	Notable Design Decisions	5
5	Execution View (Component & Connector Diagram)	5
5.1	Request Processing Flows	5
5.1.1	Upload flow	5
5.1.2	Manual flow	6
5.1.3	History flow	6
6	Deployment View	6
6.1	Deployment Pipeline	6
6.2	Infrastructure Requirements	7
7	Data Model	7
8	Conclusion	8

1. Introduction

Hissab (Arabic for calculation) is a component-based web application built on the Jakarta EE 10 platform. It is designed in an educational context to help primary school students — guided by their teacher — verify arithmetic results by scanning a list of operations from a class workbook, uploading the scanned image or a PDF, and having the system evaluate each expression automatically.

The application follows a *component-based architecture* in which the arithmetic calculator, originally developed as a standalone utility, has been promoted to a reusable **CALC** component implemented as a Stateless EJB. This component is then integrated into the broader **HISSAB** system alongside an OCR component and a persistence component.

1.1. Key Features

- **OCR mode** — upload a PNG, JPEG, or PDF file; Tesseract 5 extracts arithmetic expressions and any student-provided results from the scanned document, evaluates each expression via the CALC component, and verifies whether the provided answers are correct.
- **Manual mode** — type an expression and an optional expected answer, the system evaluates the expression and verifies the provided answer against the computed result.
- **Step-by-step breakdown** — each computation is decomposed into individual binary operations and displayed to the student.
- **Answer verification** — provided answers are compared against the computed result within an epsilon of 10^{-9} .
- **History & export** — all traces are stored in PostgreSQL, searchable by keyword, and downloadable as CSV.

2. Environment & Technology Stack

The following table summarises the software stack and the exact versions required to build and run the application, as defined in the project setup guide.

Layer	Technology	Version
Language	Java (OpenJDK)	21 LTS
Platform	Jakarta EE	10.0
Application Server	GlassFish	8.0.1
Web Layer	Jakarta Servlet + JSP	6.0
Business Layer	EJB Stateless	Jakarta EJB 4.0
Persistence	EclipseLink JPA	3.0 · JTA
Database	PostgreSQL	14+
JDBC Driver	postgresql	42.7.3
OCR Engine	Tesseract	5.3.4 (fra + eng)
OCR Java Bridge	Tess4j	5.11.0
Build Tool	Apache Maven	3.8.7
Testing	JUnit Jupiter	5.10.2

Table 1: Technology stack and versions

2.1. Infrastructure Setup

The environment is provisioned in the following order (see the project setup guide for the full command listing):

1. **Java 21** is installed via `apt` and `JAVA_HOME` is exported. Both `java` and `javac` must report version 21.
2. **Maven** is installed via `apt`. Running `mvn -version` must confirm it uses Java 21; a mismatch in `JAVA_HOME` is the most common setup error.
3. **GlassFish 8.0.1** is downloaded from the Eclipse Foundation, extracted to `/opt/glassfish8`, and `GLASSFISH_HOME` is exported. The domain listens on port 8080 (HTTP) and 4848 (admin console).
4. **PostgreSQL** is installed and a dedicated database is created:

```

1 CREATE USER hissab WITH PASSWORD 'hissab123';
2 CREATE DATABASE hissabdb OWNER hissab;
3 GRANT ALL PRIVILEGES ON DATABASE hissabdb TO hissab;
```

Listing 1: Database provisioning

5. The **PostgreSQL JDBC driver** (`postgresql-42.7.3.jar`) is copied into `$GLASSFISH_HOME/domains/domain1/lib/` before GlassFish is started.
6. The **JDBC connection pool** `HissabPool` and resource `jdbc/hissabDS` are registered via `asadmin`. This JNDI name is referenced in `persistence.xml`.
7. **Tesseract 5** is installed with the `fra` and `eng` language packs. The `tessdata` directory (typically `/usr/share/tesseract-ocr/5/tessdata/`) is configured in the OCR EJB.

3. Architecture Overview

The application follows a classic three-tier Jakarta EE architecture with a component-based decomposition of the business layer.

- Presentation tier** — JSP views (`index.jsp`, `result.jsp`, `history.jsp`, `error.jsp`) served through two HTTP servlets.
- Business tier** — three Stateless EJBs: `CalcEJB` (CALC component), `OcrEJB`, and `TraceEJB`, each injected via `@EJB` into the servlets.
- Data tier** — a single JPA entity (`ComputationTrace`) mapped to the `computation_trace` PostgreSQL table, managed by EclipseLink with DDL auto-generation (`create-or-extend-tables`).

4. Development View (Class Diagram)

Figure 1 presents the static structure of the application — the classes, interfaces, and their relationships.

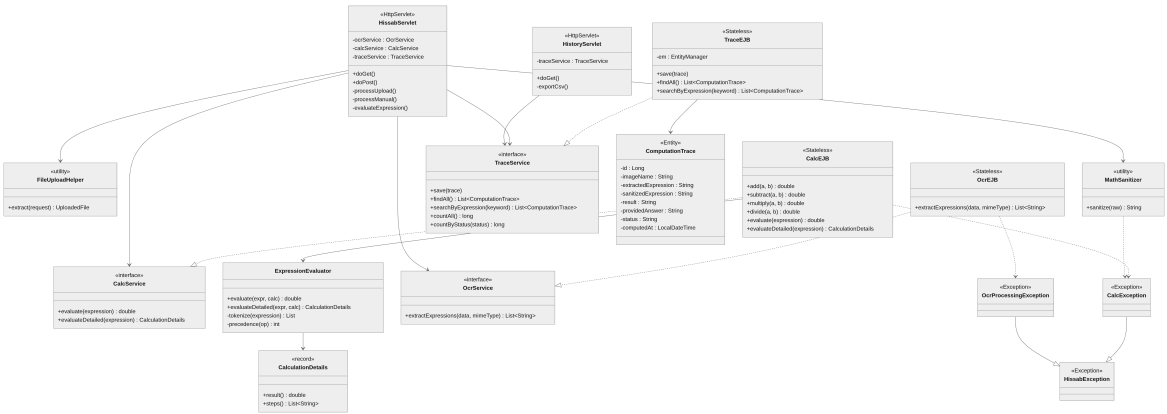


Figure 1: Class diagram — development view

4.1. Package Structure

Package	Responsibility
<code>dz.hissab.servlet</code>	HTTP entry points, request routing, view forwarding
<code>dz.hissab.calc</code>	CALC component: arithmetic EJB + Shunting-Yard engine
<code>dz.hissab.ocr</code>	Tesseract 5 OCR integration
<code>dz.hissab.trace</code>	Computation history persistence (JPA)
<code>dz.hissab.util</code>	Expression sanitization utility
<code>dz.hissab.exception</code>	Typed checked exception hierarchy

Table 2: Package responsibilities

4.2. Notable Design Decisions

- Each EJB exposes a `@Local` interface (`CalcService`, `OcrService`, `TraceService`), decoupling the servlet layer from concrete implementations.
- `ExpressionEvaluator` implements the *Shunting-Yard algorithm* to handle operator precedence and parentheses without any third-party expression parser.
- `MathSanitizer` normalizes Unicode math symbols ($\times$, $\div$, $-$) to their ASCII equivalents before evaluation, ensuring robustness against OCR character substitutions.
- `ComputationTrace` stores `breakdownSteps` as a `@Transient` field: the step list is produced at runtime and is not persisted to the database.
- The exception hierarchy (`HissabException` $\leftarrow$ `CalcException`, `OcrProcessingException`) provides typed error propagation from every layer up to the servlet’s user-friendly error handler.

5. Execution View (Component & Connector Diagram)

Figure 2 shows the runtime view — the active components and the connectors through which they communicate during execution.

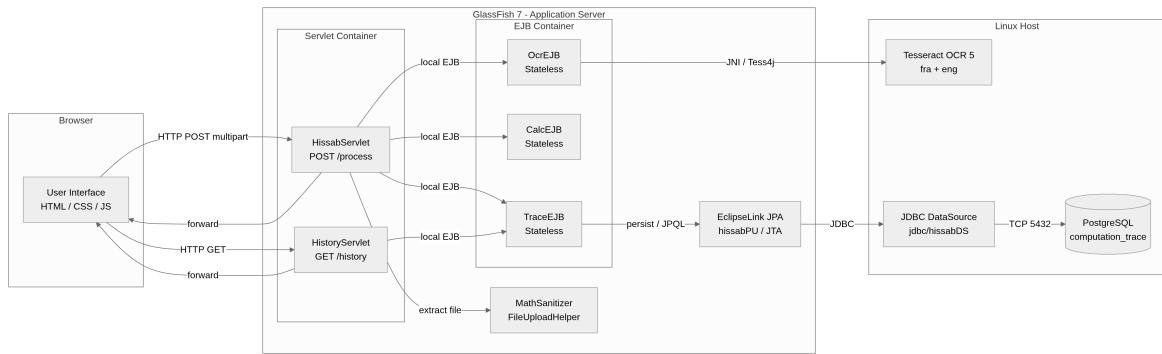


Figure 2: Component & Connector diagram — execution view

5.1. Request Processing Flows

5.1.1. Upload flow

The browser submits a multipart `POST` to `/process`. `HissabServlet` delegates file extraction to `FileUploadHelper`, then calls `OcrEJB`, which writes a temporary file and invokes the Tesseract 5 engine via the `Tess4j` JNI bridge (languages: `fra+eng`). The OCR component extracts arithmetic expression lines as well as any student-provided results present in the scanned document. The extracted expressions are sanitized by `MathSanitizer` and evaluated by `CalcEJB` (the `CALC` component), which uses `ExpressionEvaluator` internally. When a result is detected in the scanned content, the servlet compares it to the computed value using a tolerance of $\varepsilon = 10^{-9}$ and determines whether the answer is correct. The resulting `ComputationTrace` is persisted by `TraceEJB` through `EclipseLink` over the `jdbc/hissabDS` JDBC resource, and the request is forwarded to `result.jsp`.

5.1.2. Manual flow

Identical to the upload flow except that `OcrEJB` is not invoked; the expression comes directly from the HTTP request parameter. If the student supplies an expected answer, the servlet compares it to the computed result using a tolerance of $\varepsilon = 10^{-9}$ and returns a SUCCESS or ERROR status accordingly.

5.1.3. History flow

`HistoryServlet` calls `TraceEJB` for a full list or a keyword-filtered subset, collects aggregate statistics (total, success, error, verified counts), and forwards to `history.jsp`. A second URL pattern (`/history.csv`) streams all traces as a downloadable CSV file with CSV-injection protection.

6. Deployment View

Figure 3 illustrates the physical deployment of the application on the target machine.

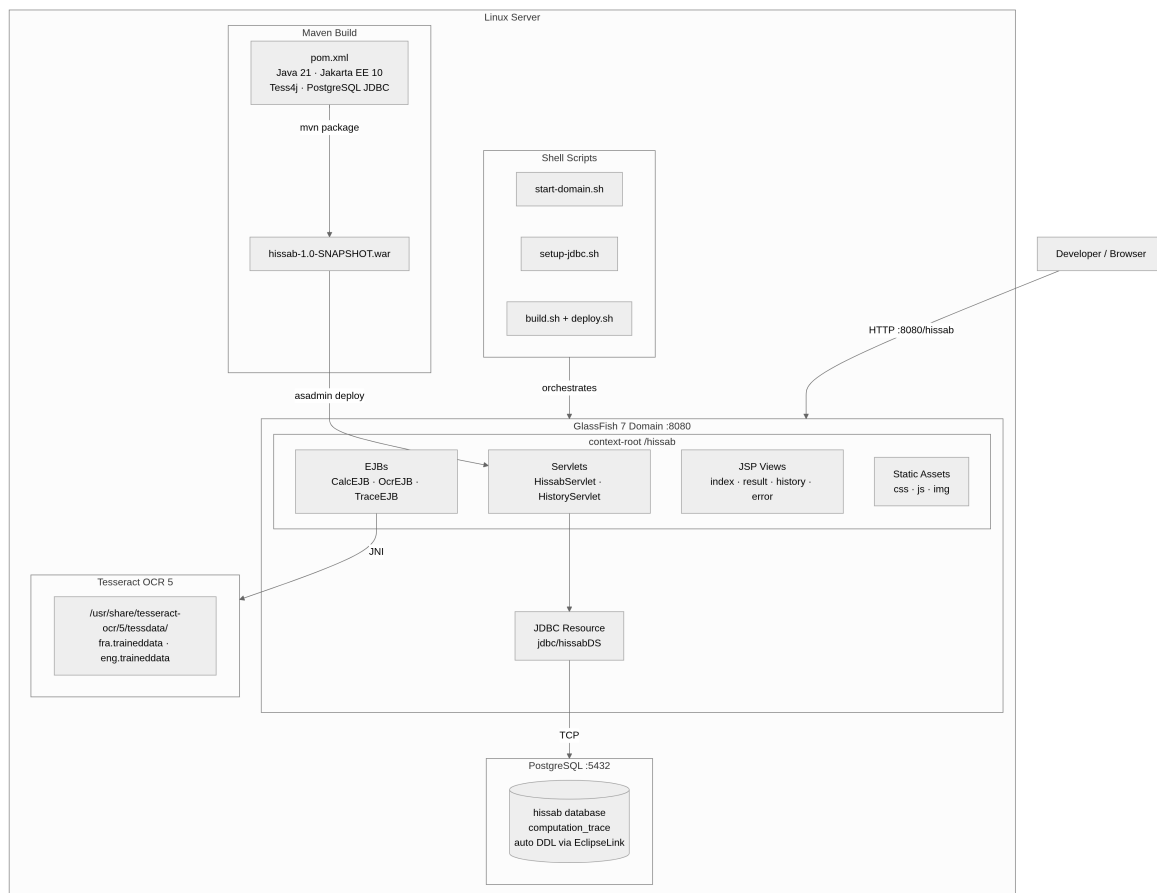


Figure 3: Deployment diagram

6.1. Deployment Pipeline

The `scripts/` directory provides a fully scripted deployment pipeline:

1. `start-domain.sh` — starts the GlassFish 8.0.1 domain (`asadmin start-domain`).

2. `setup-jdbc.sh` — creates the `HissabPool` connection pool pointing to `hissabdb` on `localhost:5432`, then registers the JNDI resource `jdbc/hissabDS`.
3. `build.sh` — runs `mvn clean package` to produce `hissab-1.0-SNAPSHOT.war`.
4. `deploy.sh` — deploys the WAR to GlassFish at context root `/hissab` via `asadmin deploy`.
5. `smoke-test.sh` — sends a basic HTTP request to `http://localhost:8080/hissab` to verify the application is live.

6.2. Infrastructure Requirements

Component	Detail	Port / Path
GlassFish 8.0.1	Application server	8080 (HTTP), 4848 (admin)
PostgreSQL	Database <code>hissabdb</code> , user <code>hissab</code>	5432
JDBC Driver	<code>postgresql-42.7.3.jar</code> in <code>domain1/lib/</code>	—
Tesseract 5	With <code>fra + eng</code> language packs	—
tessdata	<code>/usr/share/tesseract-ocr/5/tessdata/</code>	—
Java 21	<code>JAVA_HOME</code> must be set	—

Table 3: Infrastructure requirements summary

7. Data Model

A single entity, `ComputationTrace`, is persisted in the `computation_trace` table. EclipseLink creates or extends this table automatically on first deployment (`create-or-extend-tables`).

Column	SQL Type	Description
<code>id</code>	<code>BIGSERIAL</code>	Auto-generated primary key
<code>image_name</code>	<code>VARCHAR(255)</code>	Source filename or “ <i>manual</i> ”
<code>extracted_expression</code>	<code>VARCHAR(500)</code>	Raw line from Tesseract output
<code>sanitized_expression</code>	<code>VARCHAR(500)</code>	Normalized ASCII expression
<code>result</code>	<code>VARCHAR(100)</code>	Computed result or error message
<code>provided_answer</code>	<code>VARCHAR(100)</code>	Student-supplied answer (nullable)
<code>status</code>	<code>VARCHAR(20)</code>	<code>SUCCESS</code> or <code>ERROR</code>
<code>computed_at</code>	<code>TIMESTAMP</code>	Set at object construction, NOT NULL

Table 4: Table `computation_trace` — column definitions

8. Conclusion

Hissab demonstrates a well-structured, component-based Jakarta EE 10 application in which the arithmetic calculator has been encapsulated as a reusable EJB component (CALC) and integrated into the broader HISSAB system. The separation of concerns across the servlet, EJB, and persistence layers, combined with the scripted deployment pipeline targeting GlassFish 8.0.1 and PostgreSQL, makes the solution reproducible and easy to demonstrate. The Tesseract OCR integration provides a practical bridge between physical school workbooks and automated computation, directly addressing the educational use case defined in the activity specification.